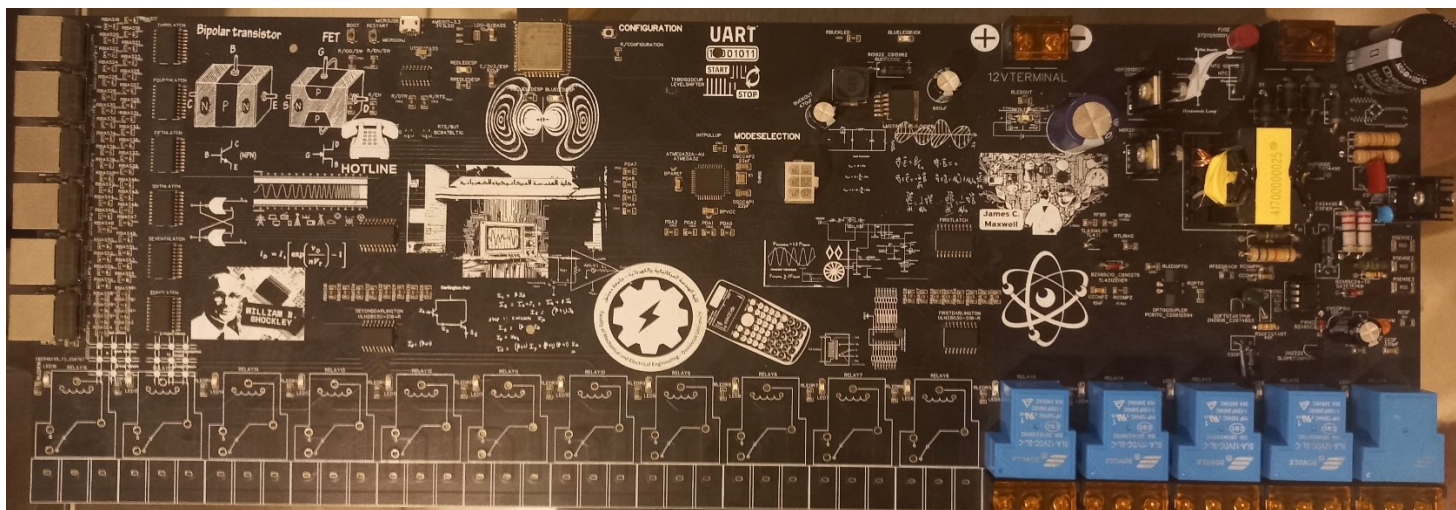
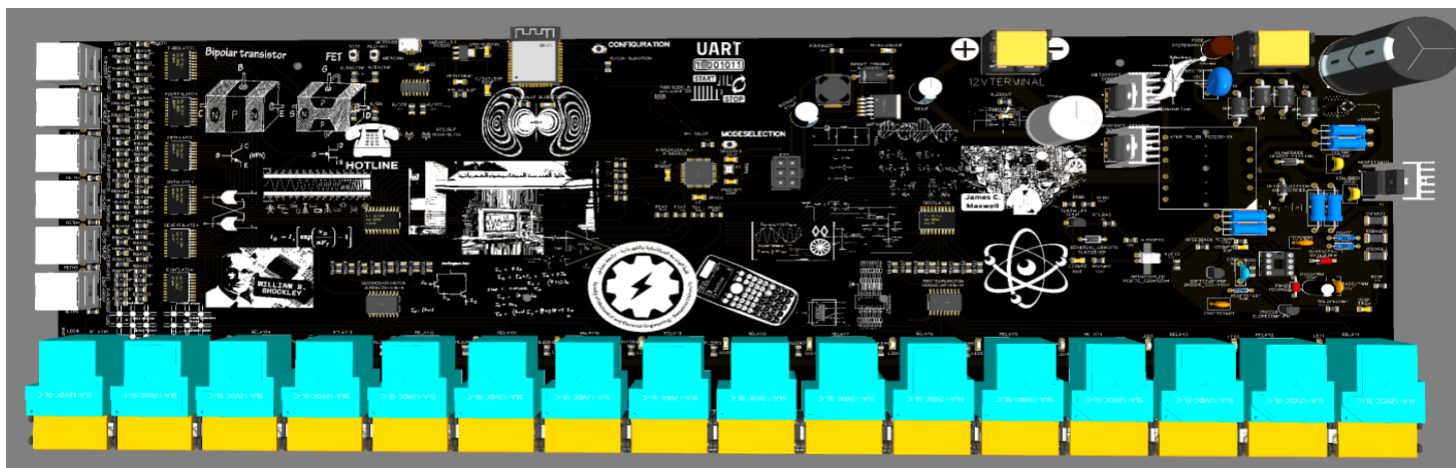


Scalable and Configurable Electronic Control System Based on the Internet of Things

A project awarded Second Place in the Electronics and Communication Engineering Department's representation at the Faculty of Mechanical and Electrical Engineering Exhibition (FMEE) 2025.



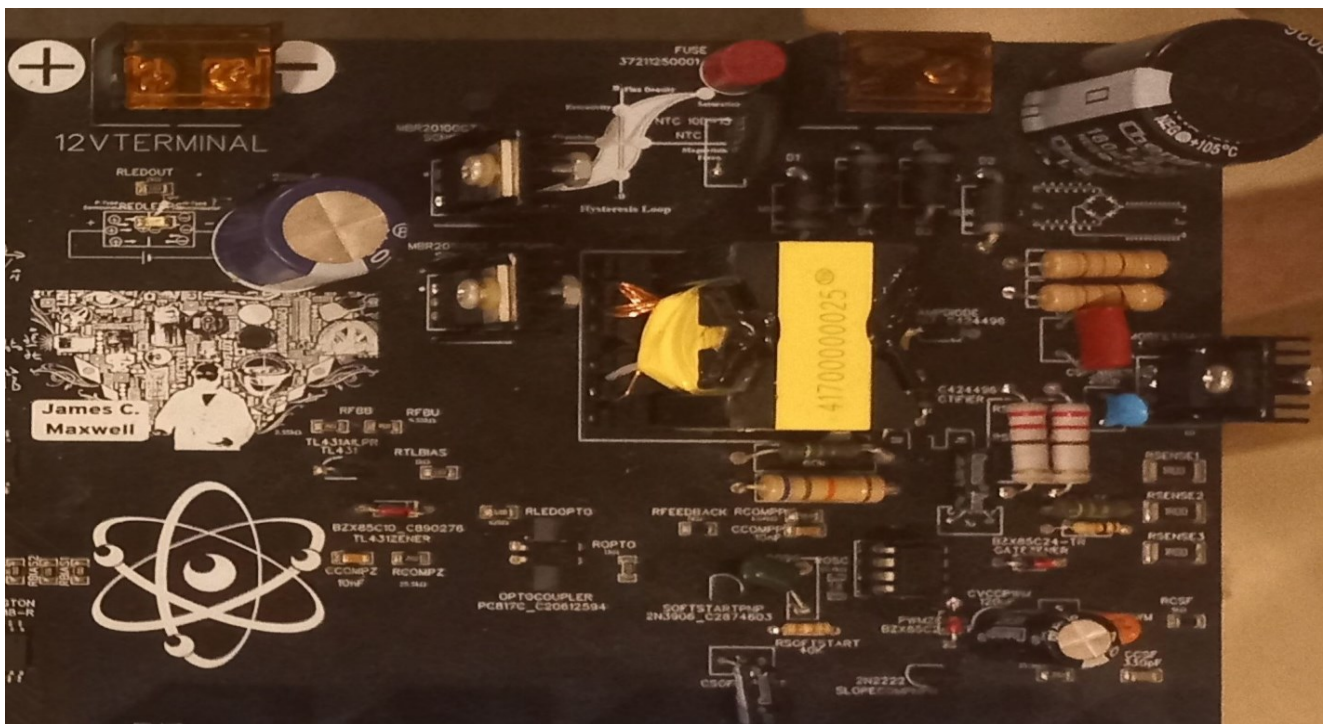
This project is a load-controlling system-on-board that operates primarily via the internet using the Thingier.io application. It consists of 16 single-pole double-throw (SPDT) relays, each capable of switching an electrical load rated at 220V and up to 30A. Control is achieved through a custom-built dashboard within the application.

Objectives.

- A Minimum Viable Product (MVP) with low engineering cost, is easily adaptable to specific market needs.
- Resources and Potentials that support adding and adjusting most of the system.
- Capability of supporting up to extra 48 Relays by adding extension boards that can be provided by the system manufacturer.
- On-Board Power Supply system of 90W (Flyback DC to DC Converter) that can feed the controlling and relays circuitry as well as any future-added features depending on the market research.
- Easy and product-level configuration method.

System Architecture.

- ❖ 100VAC to 260VAC Input and on-board 12VDC and 7.5A Output.

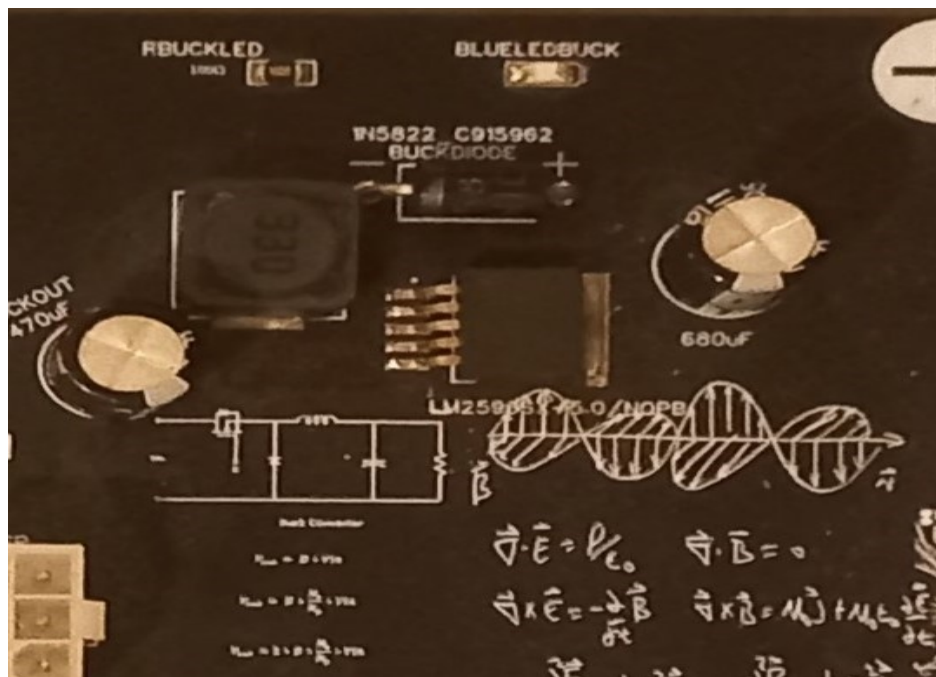


This part of the system is responsible for taking in the AC Power and converting it to DC power so that it can feed all the components on the board.

It is a Flyback Converter designed and built with the Texas Instruments' UC3842 Current-Mode PWM Controller, along with a cheap transformer that I reverse-engineered to comply with the design.

The power supply circuit was tested under up to 45W load. It showed good response, stability and thermal signature except for a single resistor that must be replaced with a higher valued one.

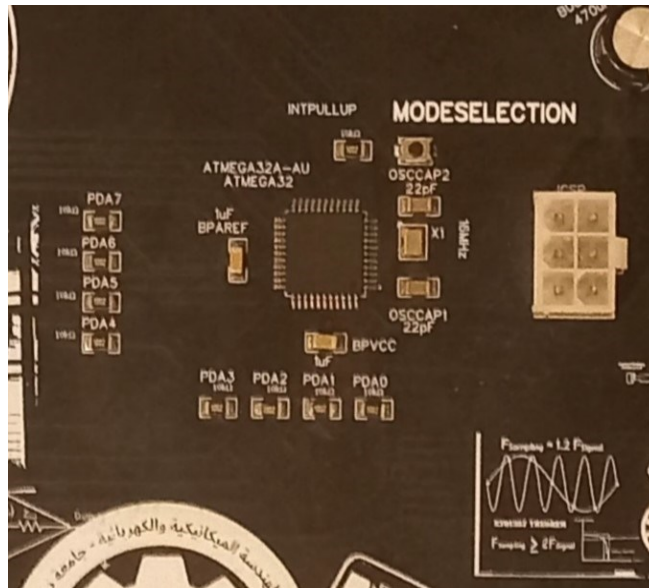
- ❖ 6VDC to 36VDC Input and 5VDC with 3A Output Step Down Converter (Buck Converter).



This part is responsible for feeding the Micro Controller Units MCUs (AVR ATmega32 and Esp32 Chip).

It is designed with the fixed-voltage version of Texas Instruments LM2596 IC and its provided design sheets.

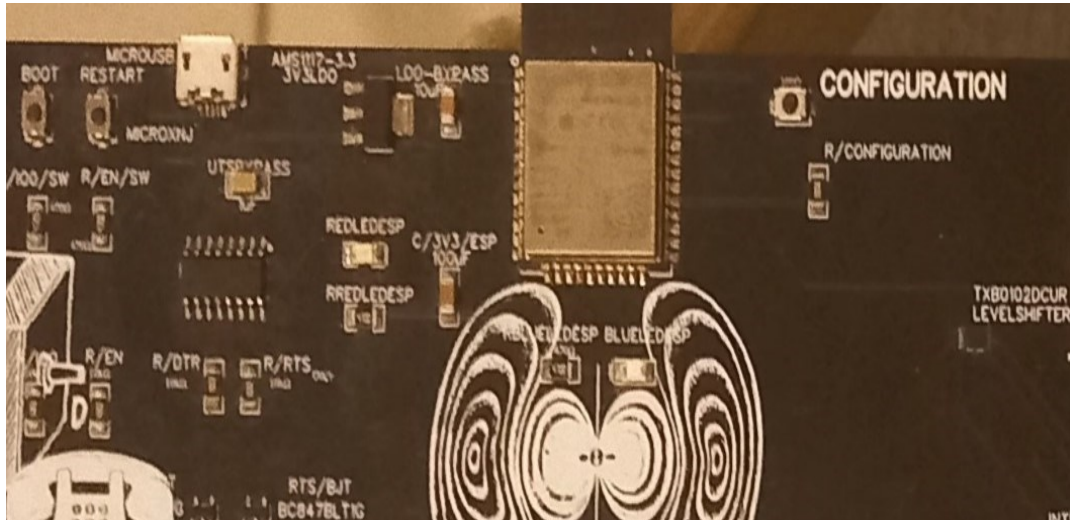
- ❖ AVR ATmega32 MCU: The heart of the project. Chosen for its versatility. Programmed with C in Atmel Studio using an Arduino Uno as an STK500 programmer by support of AVR Dude. 2 Ports are dedicated to the 64 possible relays controlling. Consequently, 2 Ports are still available for upgrading features.



The MCU runs at 16MHz External Crystal Oscillator, has an ISP (In-System Programmer) Connector. With Pull-Down resistors to suppress floating-effect.

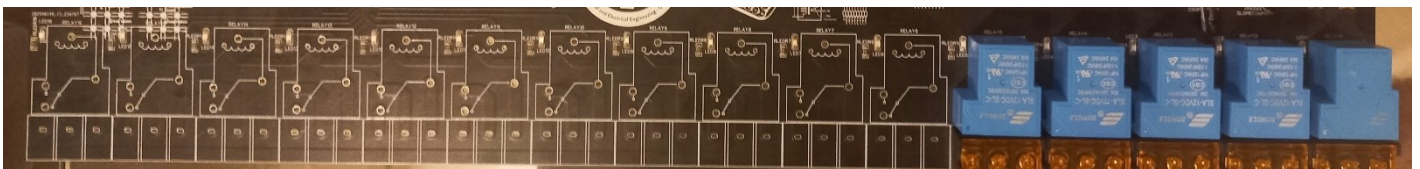
- ❖ Espressif ESP32 Node MCU: A well-known, cost-effective, and widely used chip in IOT applications. This chip communicates with Thinger.io via the internet service available in the controlled facility. The chip can be configured via a button called "configuration". When pressed, it starts an access point that can be reached with a cell phone or any smart device. Afterwards, the user goes to any browser and types the ESP32 IP address. Then, a user interface appears, which can accept the SSID and Password of the existing WiFi network and store them in the permanent memory of the chip.

Then it connects to the WiFi network, communicates with the platform, and sends any received orders from the platform to the ATmega32 MCU in a UART-based stable protocol that I designed myself.



Apparently, I designed a custom ESP32 module on the board. It is better and cheaper than using an already off-shelf ESP32 module. It can be programmed via the USB port.

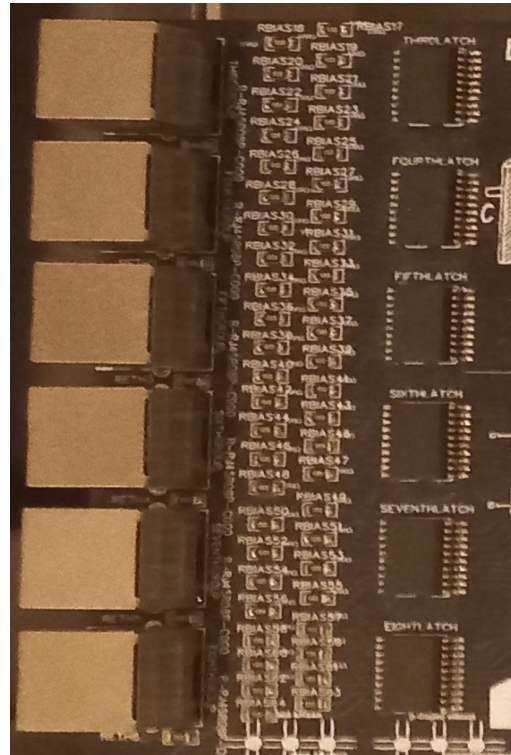
❖ Inbuilt 2x8 Relay Arrays.



In Application, I soldered only 5 Relays for the purpose of testing. However, in the product they must be 16. Each Relay has a Common, NC, and NO contacts.

For example, a line that's supplying a water pump can be easily attached to the NO contact, then the Common contact supplies the pump. Hence, when the user turns on the relay, it conducts electricity into the pump and vice versa.

❖ Extension boards output.



Provided that the already inbuilt relay arrays are not sufficient for the user, each of these RJ45 connectors can control an 8-relays array extension which can be easily provided by the manufacturer. The power must be drawn from the board's 12V terminal. In other words, more extensions: more versatility.

❖ Printed Circuit Board (PCB)

Confirm Product Detail

Top

Bottom

Is the production file ok to be proceeded?

☒ Yes, please proceed to production

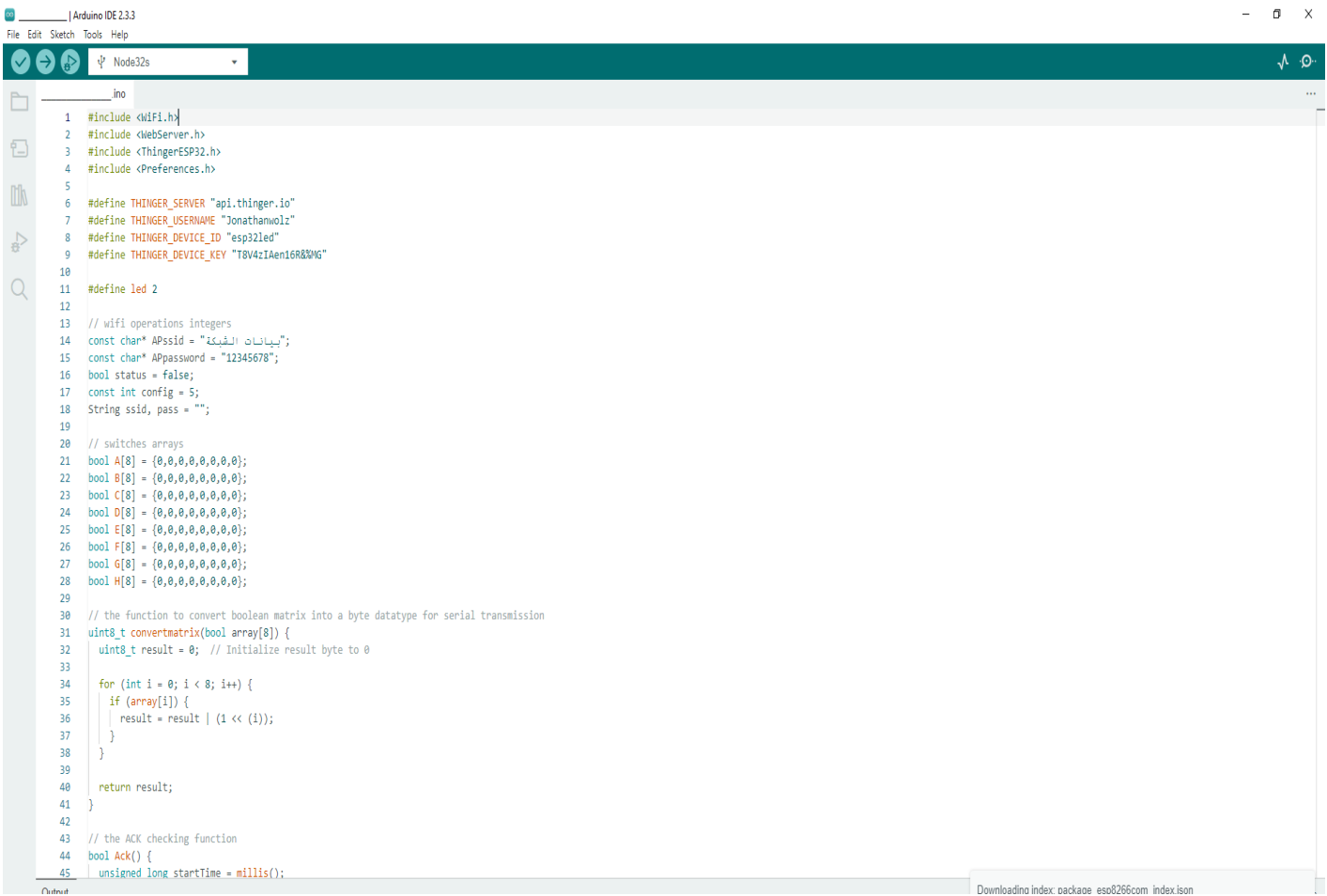
☐ Not good, modification needed

Submit

The PCB was designed and laid out in EasyEDA, a free platform that is integrated with a parts supplier-LCSC, and JLCPCB factory. A complete Chinese ecosystem that provides all the required features for a full-stack embedded systems developer. The PCB order was made at JLCPCB, the parts were ordered from LCSC, and the soldering process was done locally by hands.

Software & Firmware.

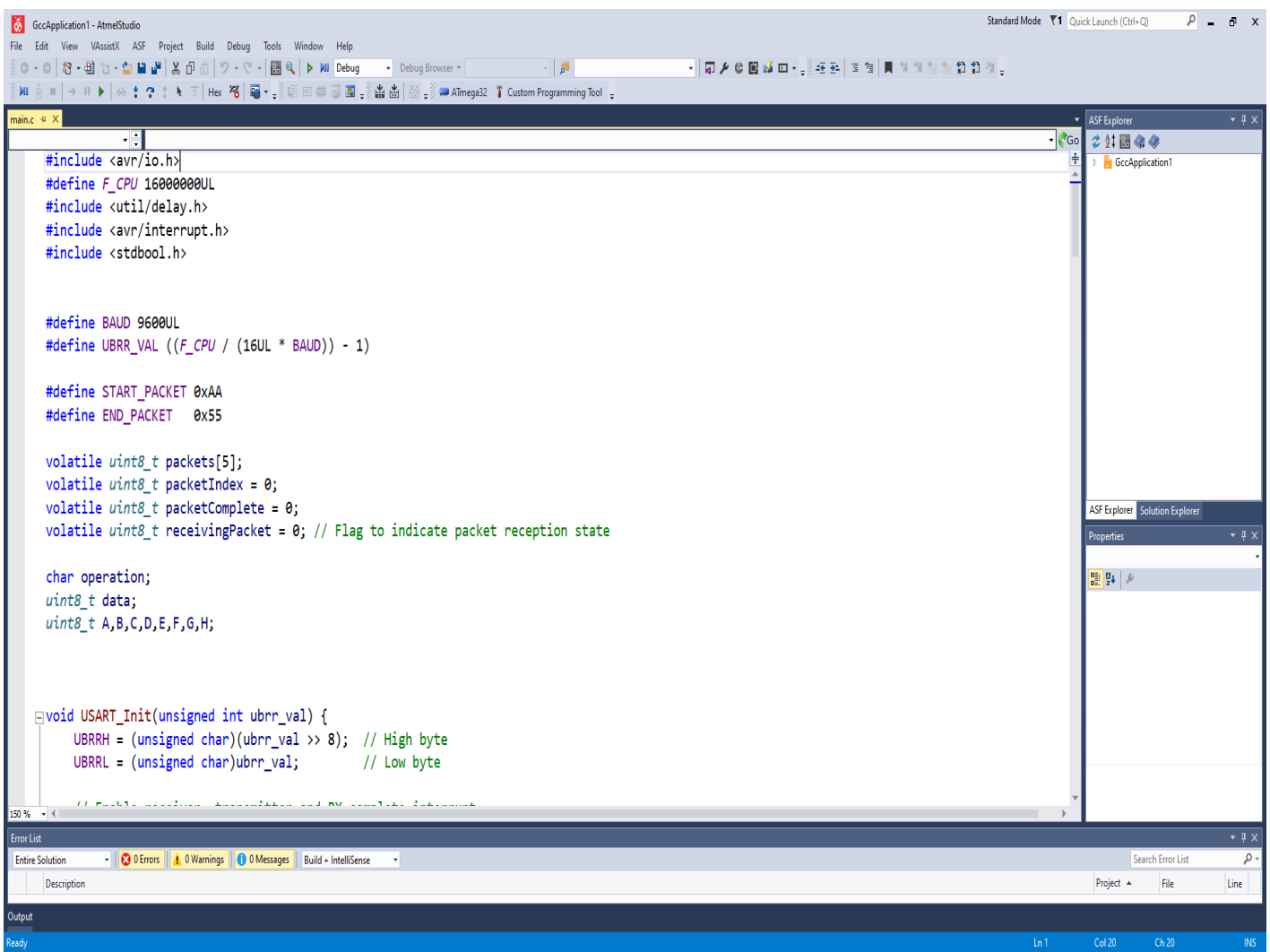
❖ C++ was used to code the ESP32 with Arduino IDE.



```
1 #include <WiFi.h>
2 #include <WebServer.h>
3 #include <ThingiverseESP32.h>
4 #include <Preferences.h>
5
6 #define THINGER_SERVER "api.thinger.io"
7 #define THINGER_USERNAME "Jonathanwolz"
8 #define THINGER_DEVICE_ID "esp32led"
9 #define THINGER_DEVICE_KEY "T8V4zIAen16R&Wg"
10
11 #define led 2
12
13 // wifi operations integers
14 const char* APssid = "بيانات الشبكة";
15 const char* APPassword = "12345678";
16 bool status = false;
17 const int config = 5;
18 String ssid, pass = "";
19
20 // switches arrays
21 bool A[8] = {0,0,0,0,0,0,0,0};
22 bool B[8] = {0,0,0,0,0,0,0,0};
23 bool C[8] = {0,0,0,0,0,0,0,0};
24 bool D[8] = {0,0,0,0,0,0,0,0};
25 bool E[8] = {0,0,0,0,0,0,0,0};
26 bool F[8] = {0,0,0,0,0,0,0,0};
27 bool G[8] = {0,0,0,0,0,0,0,0};
28 bool H[8] = {0,0,0,0,0,0,0,0};
29
30 // the function to convert boolean matrix into a byte datatype for serial transmission
31 uint8_t convertmatrix(bool array[8]) {
32   uint8_t result = 0; // Initialize result byte to 0
33
34   for (int i = 0; i < 8; i++) {
35     if (array[i]) {
36       result = result | (1 << (i));
37     }
38   }
39
40   return result;
41 }
42
43 // the ACK checking function
44 bool Ack() {
45   unsigned long startTime = millis();
46 }
```

Download index: package es08266com index.json

- ❖ C language was used to code the ATmega32 MCU with Atmel Studio 7.0. The code was injected into the MCU via an Arduino Uno as an STK500 programmer with the help of AVRDUDE. It would have been more professional to use a standard programmer, but for research cost-effectivity I made use of the available materials.



The screenshot displays the Atmel Studio 7.0 IDE interface. The main window shows a C program named 'main.c' with the following code:

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdbool.h>

#define BAUD 9600UL
#define UBRR_VAL ((F_CPU / (16UL * BAUD)) - 1)

#define START_PACKET 0xAA
#define END_PACKET 0x55

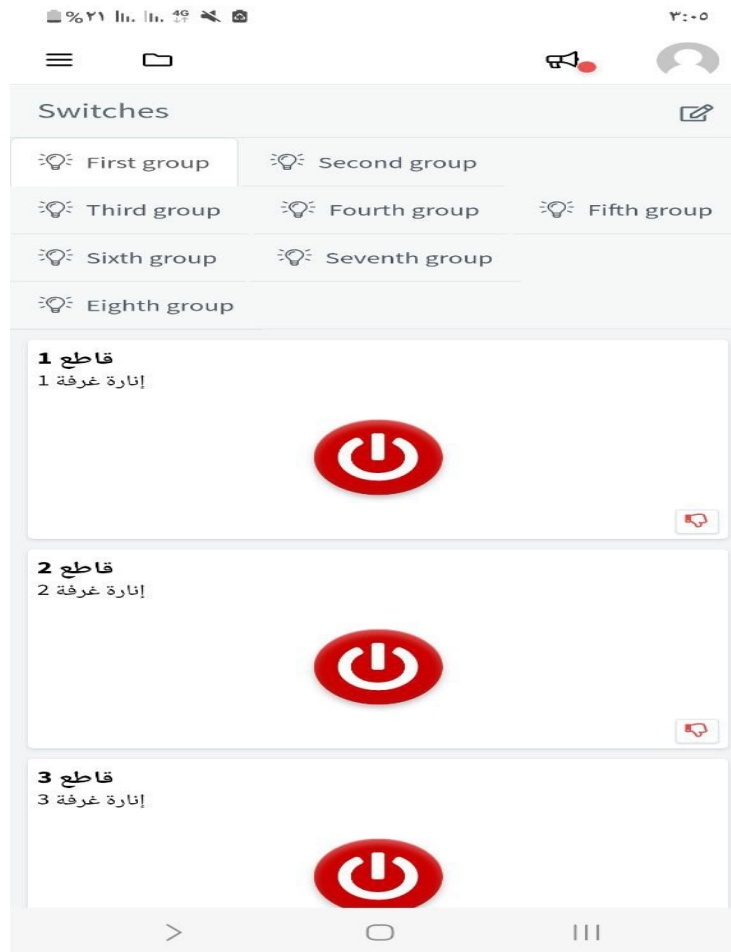
volatile uint8_t packets[5];
volatile uint8_t packetIndex = 0;
volatile uint8_t packetComplete = 0;
volatile uint8_t receivingPacket = 0; // Flag to indicate packet reception state

char operation;
uint8_t data;
uint8_t A,B,C,D,E,F,G,H;

void USART_Init(unsigned int ubrr_val) {
    UBRRH = (unsigned char)(ubrr_val >> 8); // High byte
    UBRRL = (unsigned char)ubrr_val;         // Low byte
    // Enable receiver, transmitter and RX complete interrupt
}
```

The IDE interface includes a menu bar (File, Edit, View, Assistant, ASF, Project, Build, Debug, Tools, Window, Help), a toolbar, and a status bar at the bottom showing 'Ready', 'Ln 1', 'Col 20', 'Ch 20', and 'INS'. The right sidebar contains the 'ASF Explorer' and 'Solution Explorer' panels, and the bottom panel shows the 'Error List' and 'Output' windows.

- ❖ Thinger.io as the platform which I made the system compatible with and can be linked to with few steps. If further development occurs with the help of software embedded systems engineer, an independent application for the product can be easily made because the system works on the raw-data level.

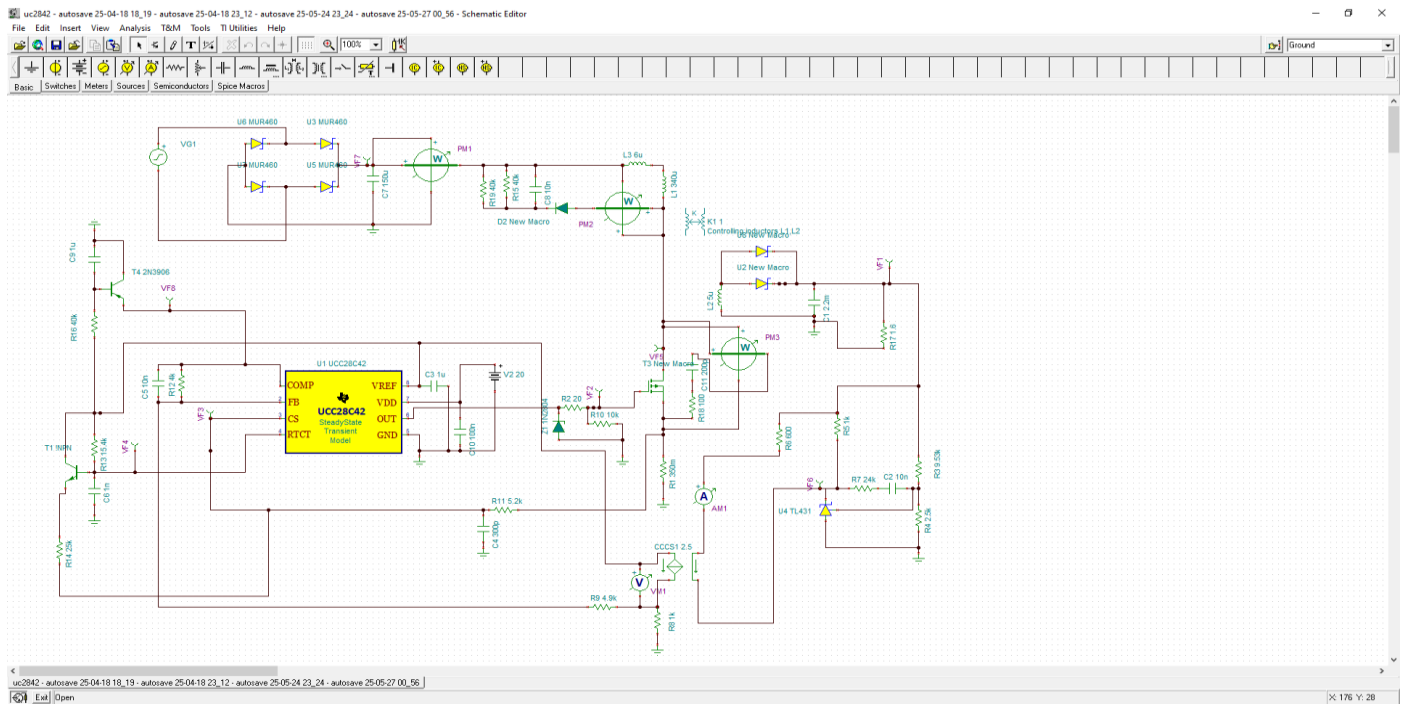


In my approach, the user first creates an account on the platform, copies the username, device id, and the device key, then pastes them in the Configuration portal of the system.

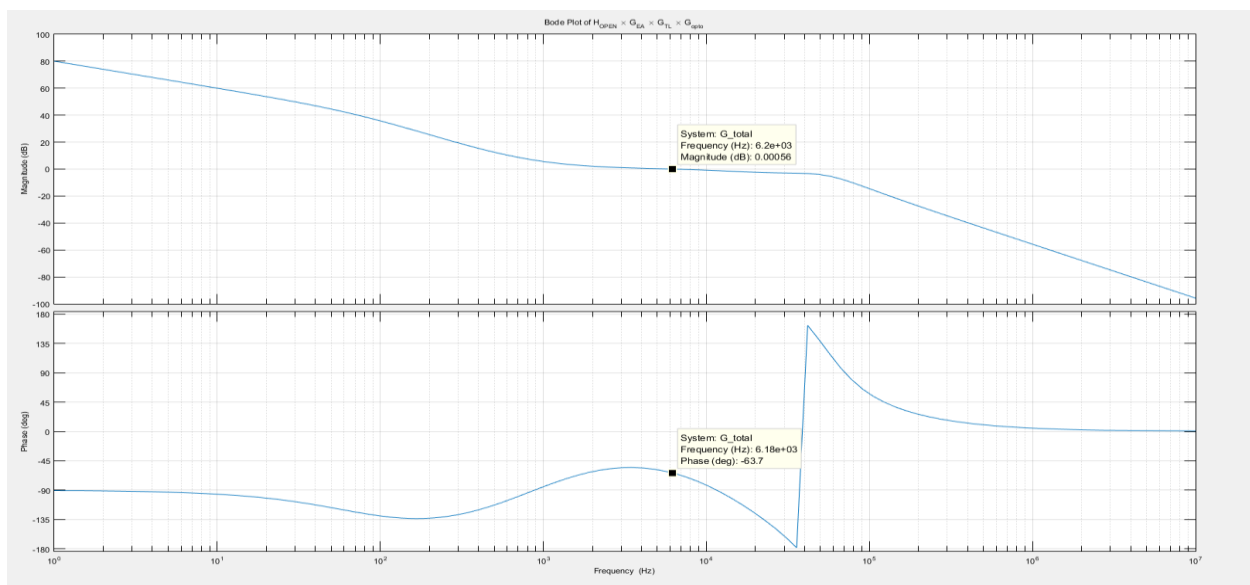
The ESP32 must connect to the platform at this stage, hence, the user can create a dashboard and name it like they want. Eventually, the user will have the ability to control any relay in the system through the Thinger.io application.

Simulation and Analysis.

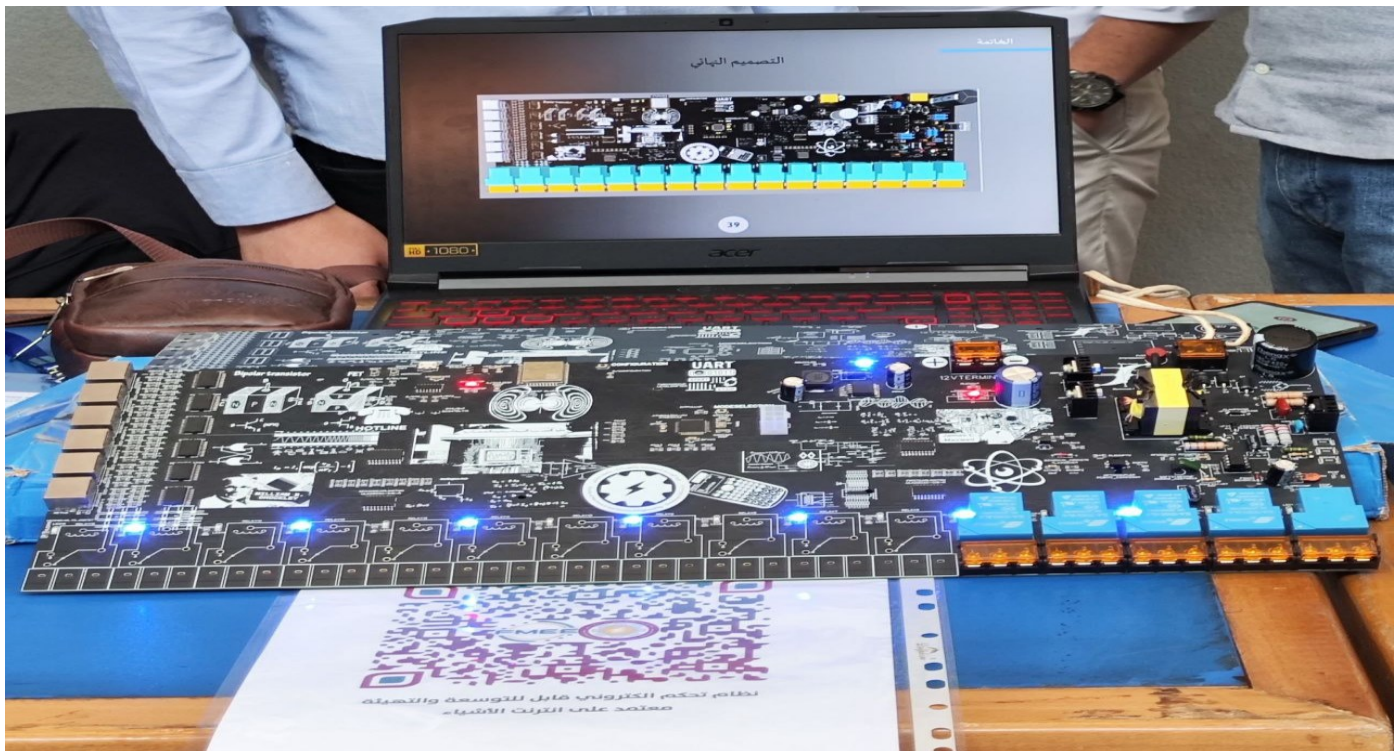
- Tina TI was used as the main simulation software. After the mathematical calculations of the Flyback Converter required components, I simulated the circuit in Tina TI because it is the only software that can read the encrypted PSPICE Model of the integrated circuit UC3842.



- MATLAB was used to calculate the RLED resistor in the feedback loop as well as the Bode Plot of the Flyback Converter with the purpose of achieving stability.



Testing and exhibiting.



The project has run for 9 continuous hours in each of the 3 days of the FMEE Exhibition.

Everything was fine, no issues reported unless one resistor with a high thermal dissipation.

However, this can be fixed with replacing it by a larger resistor or dividing its value over 2 separate ones, taking into account that its value was underestimated because of the absence of laboratory.

Challenges & Solutions.

- Each board of this product can cost up to 100 USD if manufactured by low quantities. However with larger quantities, the price of the production can be optimized to go as low as 40-60 USD.
- This model was originally developed for educational purposes; still, the infrastructure of a competitive product has been developed within this project. More market-research can lead to a better version that can be a versatile option for consumers.

- Bulkiness of the board is a main constraint, so a more compact design will be an observable advance. For Instance, I can lessen the number of inbuilt relays to 8, or separate the controlling and supplying circuitry from the relay arrays part, which can strengthen the signal integrity over the board.

Applications.

This product offers an effective solution to bridge the gap between house-oriented IoT systems, which typically provide a limited number of control channels, and factory-oriented solutions such as PLCs, which are often expensive and over-engineered for applications like farms. For customers with medium-scale farms requiring a reliable control system, the next version of this product could be both budget-friendly and competitive with Chinese alternatives.

Conclusion.

This version of the project is primarily educational. However, during its design, I intentionally reserved space and resources to allow for the integration of additional features. I personally prefer not to predetermine these features, as I believe an engineer's role is to solve real problems rather than invent them. Consequently, functionalities such as metering, sensing, a more compact design, and decision-making capabilities can be incorporated depending on the needs of the target market.